Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1113 Sistemas Empotrados
Prof. Dr.-Ing. Jeferson González Gómez

Taller 8: Sistemas de Tiempo Real con Zephyr RTOS

Expropiación, Prioridades e Inversión de Prioridades sobre QEMU

1. Introducción

En el Taller 06 el firmware bare-metal consistía en un único flujo de ejecución: `main()` inicializaba el hardware y luego entraba en un `while(1)` que, a lo sumo, esperaba eventos con `wfi`. Esa estructura es adecuada para tareas simples, pero la mayoría de sistemas empotrados reales deben atender **múltiples actividades concurrentes** con distintos requisitos de urgencia: leer un sensor cada 10 ms, procesar comandos por UART cuando lleguen, parpadear un LED de estado cada segundo, todo en el mismo procesador. Escribir esa concurrencia a mano, intercalando manualmente el código de cada actividad dentro de un único `while`, se vuelve inmanejable rápidamente.

Un **sistema operativo de tiempo real** (RTOS, *Real-Time Operating System*) resuelve este problema proveyendo un **calendarizador** (*scheduler*) que reparte el procesador entre múltiples **hilos** (*threads*) de forma determinística, según una política de prioridades. A diferencia de un sistema operativo de propósito general (como Linux, visto en los Talleres 4–5), un RTOS está diseñado para que el **comportamiento temporal sea predecible**: la pregunta central no es solo "¿el programa da la respuesta correcta?" sino "¿la da **a tiempo**, siempre, incluso en el peor caso?".

Este taller usa **Zephyr**, un RTOS de código abierto de la Linux Foundation con soporte oficial para RISC-V, para explorar dos conceptos centrales de la calendarización en tiempo real: la **expropiación** (*preemption*; cómo un hilo de mayor prioridad interrumpe a uno de menor prioridad) y la **inversión de prioridades** (un problema clásico que puede romper las garantías de tiempo real si no se maneja correctamente). El taller corre íntegramente sobre **QEMU** (emulación de un procesador RISC-V genérico, `qemu_riscv64`), sin requerir una tarjeta física: los conceptos de calendarización que interesan aquí –expropiación, prioridades, inversión de prioridades– son un comportamiento del **kernel de Zephyr** (software), no una propiedad del silicio sobre el que corre, así que se observan exactamente igual en QEMU que en hardware real. Esto además permite comparar, con las mismas herramientas (QEMU, un firmware corriendo en RISC-V), los **tres modelos de ejecución** vistos en el curso: firmware **bare-metal** sin sistema operativo (Taller 06), un **sistema operativo de propósito general** como Linux (Talleres 04–05), y ahora un **RTOS**. La Sec. 4 retoma esta comparación de forma explícita.

2. Conceptos Fundamentales

2.1. Calendarización cooperativa vs. expropiativa

En un esquema **cooperativo**, un hilo conserva el procesador hasta que él mismo decide cederlo (por ejemplo, llamando a una función de espera o de cesión explícita). Esto simplifica el razonamiento sobre secciones críticas —mientras un hilo cooperativo se ejecuta, ningún otro hilo puede interrumpirlo— pero un solo hilo mal escrito (con un bucle largo) puede bloquear indefinidamente a todos los demás.
En un esquema **expropiativo**, el calendarizador puede interrumpir un hilo en ejecución en cualquier momento para cederle el procesador a otro hilo de **mayor prioridad** que esté listo para ejecutar. Esto garantiza que las tareas urgentes se atiendan sin depender de la buena voluntad de las demás, a costa de que el programador debe proteger explícitamente cualquier recurso compartido entre hilos (con semáforos, mutex, colas, etc.), ya que la interrupción puede ocurrir en cualquier punto del código.
Zephyr soporta ambos esquemas simultáneamente: cada hilo tiene una **prioridad numérica**, donde los valores **negativos** son prioridades **cooperativas** (más altas que cualquier prioridad expropiativa, y no interrumpibles por otro hilo expropiativo) y los valores **no negativos** $(0, 1, 2, \dots)$ son prioridades **expropiativas**, donde **un número menor es una prioridad mayor**. Un hilo expropiativo de prioridad 1 puede interrumpir a uno de prioridad 5 en cualquier instante; nunca al revés.

2.2. Inversión de prioridades

Cuando dos o más hilos comparten un recurso protegido (por ejemplo, una estructura de datos protegida por un semáforo o mutex), puede ocurrir un fenómeno contraintuitivo: un hilo de **alta** prioridad termina esperando, de forma indirecta, a un hilo de **baja** prioridad, incluso cuando existe un tercer hilo de prioridad **intermedia** que no tiene nada que ver con el recurso compartido. La secuencia típica es:

1. El hilo de **baja** prioridad toma el recurso compartido.
2. El hilo de **alta** prioridad se vuelve listo, necesita el mismo recurso, y se bloquea esperando a que el hilo de baja prioridad lo libere. Hasta aquí, esto es una espera normal y acotada.
3. El hilo de prioridad **intermedia** (que **no** usa el recurso compartido) se vuelve listo y, por tener mayor prioridad que el hilo de baja prioridad, lo **expropia**.
4. Mientras el hilo intermedio ocupa el procesador, el hilo de baja prioridad no puede avanzar y, por lo tanto, no puede liberar el recurso. El hilo de **alta** prioridad queda bloqueado indirectamente por el hilo **intermedio**, con el que ni siquiera interactúa. Si esto se repite o el hilo intermedio no cede el procesador, la espera de **alta** prioridad puede volverse **indefinida** (inversión de prioridades **no acotada**).

Este no es un problema teórico: en 1997 la misión *Mars Pathfinder* de la NASA sufrió reinicios inesperados del sistema en la superficie de Marte, causados exactamente por este escenario, entre una tarea de alta prioridad de recolección de datos meteorológicos, una tarea de comunicación de prioridad intermedia y un hilo de baja prioridad que protegía un bus de información compartido con un mutex [1]. El equipo diagnosticó y corrigió el problema de forma remota, en vuelo, activando la **herencia de prioridad** en el mutex involucrado.

2.3. Herencia de prioridad

La **herencia de prioridad** (*priority inheritance*) es un protocolo que resuelve la inversión no acotada: cuando un hilo de alta prioridad se bloquea esperando un recurso que sostiene un hilo de menor prioridad, este último **hereda temporalmente** la prioridad del hilo que espera, durante todo el tiempo que sostenga el recurso. Con la prioridad heredada, ningún hilo de prioridad intermedia puede expropiarlo, por lo que el hilo de baja prioridad termina su sección crítica lo antes posible, libera el recurso, y su prioridad regresa a la normal [2].
En Zephyr, los **mutex** (`struct k_mutex`) implementan herencia de prioridad de forma automática. Los **semáforos** (`struct k_sem`), en cambio, **no** implementan herencia de prioridad: son un mecanismo de sincronización más general (no tienen noción de "dueño") y su uso para proteger un recurso compartido puede reproducir exactamente el escenario de inversión no acotada descrito arriba. La Parte 2 de este taller reproduce este contraste directamente.

3. Zephyr RTOS

3.1. ¿Qué es Zephyr?

Zephyr es un RTOS de código abierto, mantenido por la Linux Foundation, con soporte para decenas de arquitecturas (ARM, RISC-V, x86, entre otras) y cientos de tarjetas de desarrollo. A diferencia de escribir firmware bare-metal desde cero (Taller 06), Zephyr provee un kernel con calendarizador expropiativo/cooperativo configurable, primitivas de sincronización (semáforos, mutex, colas de mensajes), gestión de memoria, y un modelo de *devicetree* y *Kconfig* para describir el hardware y configurar el kernel sin tocar el código de la aplicación.

3.2. `west`: la herramienta de meta-construcción

Zephyr usa `west` para gestionar el ciclo de vida de un proyecto: descargar el código fuente del kernel y sus módulos (`west init` / `west update`), compilar una aplicación para una tarjeta específica (`west build`), y cargarla o depurarla en hardware (`west flash` / `west debug`). Internamente, `west build` invoca CMake y Ninja, de forma similar a lo visto en el Taller 03.

3.3. Estructura de una aplicación Zephyr

Cada aplicación Zephyr es un directorio con, como mínimo, tres archivos:

Archivo	Rol
`CMakeLists.txt`	Declara el proyecto y qué archivos fuente se compilan, enlazando contra el kernel de Zephyr (`find_package(Zephyr)`).
`prj.conf`	Configuración de **Kconfig**: qué subsistemas del kernel se habilitan (consola, tamaño de stacks, número de prioridades, etc.), análogo a configurar el kernel de Linux antes de compilarlo.
`src/main.c`	Punto de entrada de la aplicación: `int main(void)`.

3.4. La plataforma: QEMU (`qemu_riscv64`)

Por disponibilidad de hardware, este taller usa como *board target* `qemu_riscv64`: no un SoC comercial específico, sino la máquina virtual genérica `virt` de QEMU para la arquitectura RISC-V de 64 bits, el mismo emulador QEMU usado en el Taller 06 (aunque allí con un fork propio del curso, `qemu-tec`, para modelar un periférico MMIO educativo; aquí se usa el QEMU genérico que trae el SDK de Zephyr). La máquina `virt` expone un solo *hart* RISC-V RV64GC con una UART mapeada en memoria como consola, suficiente para todos los conceptos de este taller: el calendarizador de Zephyr, la expropiación entre hilos y la herencia de prioridad son comportamiento del kernel, idéntico en QEMU y en silicio real.
A diferencia de un microcontrolador real, **no hay que cargar (*flash*) nada a una memoria física**: el comando `west build -t run` compila y, en el mismo paso, arranca QEMU con el ELF generado directamente como imagen de arranque, de forma análoga a como `./scripts/run.sh` invocaba QEMU en el Taller 06. La salida de la UART de Zephyr se conecta a la terminal donde se ejecuta ese comando, igual que en Taller 06; se sale con `Ctrl+A` seguido de `X`.

4. RTOS vs. Bare-Metal vs. Linux

El curso ha mostrado, hasta este punto, tres formas distintas de ejecutar código sobre el mismo tipo de procesador (RISC-V, emulado en QEMU): el firmware **bare-metal** del Taller 06, el **Linux embebido** construido con Yocto en los Talleres 04–05 (sobre QEMU ARM), y ahora un **RTOS** (Zephyr). Las tres comparten el mismo hardware subyacente (real o emulado) pero resuelven de forma completamente distinta la pregunta "¿quién decide qué código se ejecuta en cada instante, y qué le pasa al programador?".

4.1. Modelo de ejecución

En el Taller 06, `start.S` salta directamente a `main()`, y de ahí a `app.c`: hay **un solo flujo de ejecución**, escrito y controlado enteramente por el estudiante. No existe la noción de "hilo" ni de "calendarizador": si el código entra en un bucle largo, nada más en el sistema puede ejecutar mientras tanto (salvo una interrupción de hardware, atendida por un *handler* que el propio estudiante instaló). El **estudiante es el sistema operativo**.

En Linux (Talleres 04–05), el kernel multiplexa **procesos** (espacios de direcciones independientes, protegidos por la MMU) usando un calendarizador de propósito general (CFS u otro, según versión) que busca equidad y *throughput* agregado del sistema completo, sin garantías fuertes de cuándo exactamente correrá un proceso dado. El programador de aplicaciones casi nunca ve el calendarizador directamente: escribe programas de usuario que interactúan con el kernel a través de **syscalls** (`open`, `read`, `fork`, etc.), y el kernel decide cuándo y en qué *core* correrlos.

En Zephyr (este taller), el kernel multiplexa **hilos** dentro de un único espacio de direcciones (sin MMU por defecto: todos los hilos comparten la misma memoria, no hay aislamiento entre ellos como en procesos de Linux) usando un calendarizador **basado en prioridades estrictas**, diseñado para que el comportamiento temporal sea predecible: dado un conjunto de hilos y prioridades, se puede razonar (y en sistemas de tiempo real *duro*, demostrar matemáticamente) cuándo correrá cada uno en el peor caso. El programador llama directamente a funciones del kernel (`k_thread_create`, `k_mutex_lock`, etc.) enlazadas en el mismo binario, sin la frontera syscall/kernel de Linux.

4.2. Tabla comparativa

Aspecto	**Bare-metal (T06)**	**Zephyr RTOS (T08)**	**Linux (T04–05)**
Unidad de concurrencia	Ninguna (un solo flujo)	Hilos (`k_thread`) en un espacio de direcciones	Procesos (espacios de direcciones aislados por MMU)
Calendarizador	No existe; el programador ordena el código a mano	Prioridades estrictas, expropiativo/cooperativo, determinista	Propósito general (equidad + *throughput*), no determinista
Memoria / protección	Sin MMU; el programador controla cada dirección	Generalmente sin MMU; memoria compartida entre hilos	MMU obligatoria; aislamiento fuerte entre procesos
Interfaz programador–kernel	No aplica (no hay kernel)	Llamadas de función directas (`k_*`), enlazadas estáticamente	*Syscalls* a través de una frontera protegida (modo usuario/kernel)
Arranque	`start.S` salta a `main()` en microsegundos	Kernel de Zephyr inicializa drivers/kernel antes de `main()`, en milisegundos	Bootloader + kernel Linux + *init*, en el orden de segundos
Determinismo temporal	Total (si el programador lo garantiza)	Alto: es el objetivo de diseño explícito	Bajo: no es un objetivo de diseño de un SO de propósito general

Actividad transversal (no evaluada, recomendada)

Compare `src/threads_demo/src/main.c` de este taller con `app.c/main.c` del Taller 06: ¿qué hace Zephyr **antes** de llegar al `main()` de la aplicación que usted tuvo que hacer a mano en el firmware bare-metal (inicialización de la UART, configuración de la pila, manejo de interrupciones)? Esa diferencia es, en buena parte, lo que "paga" un RTOS a cambio de la concurrencia estructurada que provee.

5. Inicio Rápido

Todo el entorno de desarrollo (Zephyr SDK, `west`, toolchain RISC-V, QEMU) corre dentro de un contenedor Docker basado en la imagen oficial del proyecto Zephyr.

Paso 1: Construir la imagen Docker

```
docker build --build-arg UID=$(id -u) --build-arg GID=$(id -g) -t
   taller08 .
```

Este paso descarga la imagen oficial `zephyr-build` (con el Zephyr SDK) e inicializa un *workspace* de Zephyr fijado a la rama LTS `v3.7-branch`. Descarga varios GB y tarda varios minutos; solo es necesario hacerlo una vez.

Paso 2: Iniciar el contenedor

```
docker run -it --rm -v "$(pwd)":/workspace taller08
```

No se requieren banderas especiales de acceso a dispositivos: al correr sobre QEMU, no hay USB-JTAG ni puerto serial físico que exponer al contenedor.

Paso 3: Compilar y ejecutar la Parte 1

```
cd src/threads_demo
west build -b qemu_riscv64 -p always .
west build -t run
```

`west build -t run` compila (si hace falta) y arranca QEMU con el ELF resultante; no hay un paso de "flasheo" separado como en hardware real.

Paso 4: Observar la salida

La salida de la consola de Zephyr (los `printk` del firmware) aparece **directamente en la misma terminal** donde se ejecutó `west build -t run`, tal como ocurría con `./scripts/run.sh` en el Taller 06. Para salir de QEMU: `Ctrl+A` seguido de `X`.

Ciclo de desarrollo: editar `src/main.c`, luego `west build -p always .` seguido de `west build -t run`, sin necesidad de reiniciar el contenedor.

6. Parte 1: Expropiación entre hilos

El archivo `src/threads_demo/src/main.c` crea tres hilos de distinta prioridad (`ALTA`, `MEDIA`, `BAJA`) usando `k_thread_create()`. El hilo `BAJA` simula trabajo de CPU mediante una espera activa (*busy-wait*) de 1.5 segundos, mientras que `ALTA` y `MEDIA` imprimen periódicamente por UART usando `k_msleep()` entre cada impresión.
Ejecute el firmware tal como está y observe la traza serial: el objetivo es verificar, con evidencia de las marcas de tiempo impresas, que `ALTA` y `MEDIA` logran imprimir sus mensajes **durante** el

tiempo en que `BAJA` está en su espera activa, es decir, que la espera activa de un hilo de baja prioridad **no bloquea** a los hilos de mayor prioridad.

Actividades

1. Ejecute el firmware sin modificar y capture al menos 4 segundos de traza serial.
2. Agregue un cuarto hilo, de prioridad **mayor** que `PRIO_ALTA`, que imprima cada 50 ms. Documente cómo cambia el entrelazado de la salida.
3. Cambie `PRIO_BAJA` a un valor **negativo** (prioridad cooperativa) y repita el experimento. Explique, con evidencia de la traza, por qué un hilo cooperativo no puede ser interrumpido por uno expropiativo de mayor prioridad hasta que cede el procesador voluntariamente.

7. Parte 2: Inversión de prioridades

El archivo `src/priority_inversion/src/main.c` implementa el escenario descrito en la Sección 2.2: `BAJA` toma un recurso compartido y lo mantiene ocupado; `MEDIA` consume CPU periódicamente sin usar ese recurso; `ALTA` necesita el mismo recurso que `BAJA`. El macro `USE_MUTEX` en la parte superior del archivo selecciona la implementación del recurso compartido: semáforo (`USE_MUTEX 0`, reproduce la inversión) o mutex (`USE_MUTEX 1`, corrige el problema mediante herencia de prioridad).

Actividades

1. Compile y ejecute con `USE_MUTEX 0`. A partir de la traza serial, mida el tiempo transcurrido entre "`ALTA: solicitando recurso`" y "`ALTA: recurso obtenido`".

2. Cambie a `USE_MUTEX 1`, recompile (`west build -p always .`) y repita la medición.

3. Compare ambos tiempos y explique la diferencia citando explícitamente la herencia de prioridad del mutex de Zephyr.

4. En el modo mutex, agregue impresiones de `k_thread_priority_get(k_current_get())` dentro del hilo `BAJA` en tres momentos: antes de tomar el recurso, durante la sección crítica (después de que `ALTA` se bloquea), y después de liberarlo. Confirme, con la salida impresa, que la prioridad efectiva de `BAJA` sube temporalmente y luego regresa a su valor original.

8. Parte 3 (opcional, 10 % adicional): explorar SMP

Por defecto, `qemu_riscv64` arranca con un único *hart* RISC-V. QEMU puede emular varios *harts* idénticos de la misma máquina `virt` (a diferencia de una SiFive HiFive Unmatched real, con cores heterogéneos S7/U74, aquí todos los harts emulados son iguales). Esta parte es **exploratoria**: intente habilitar soporte multi-core agregando `CONFIG_SMP=y` y `CONFIG_MP_MAX_NUM_CPUS` a `prj.conf` de cualquiera de las dos aplicaciones anteriores (Zephyr ajusta automáticamente la bandera `-smp` de QEMU según `CONFIG_MP_MAX_NUM_CPUS` al invocar `west build -t run`), y documente qué ocurre.
Nota importante: a diferencia del resto del taller, este soporte **no fue verificado de antemano** en la versión de Zephyr utilizada (LTS v3.7) para `qemu_riscv64` — es posible que requiera configuración adicional, o que simplemente no esté disponible todavía para esta combinación de board y versión. Parte de la evaluación de esta sección es precisamente diagnosticar y documentar hasta dónde llega, con evidencia (mensajes de compilación, de arranque, o el error específico si no logra hacerlo funcionar). Si lo consigue, investigue y documente además: ¿cómo asigna Zephyr los hilos a los *harts* disponibles por defecto? ¿Es posible fijar un hilo a un *hart* específico (afinidad de CPU)? Demuestre, con evidencia de ejecución, la diferencia de comportamiento frente al caso de un solo *hart* de las Partes 1 y 2.

9. Evaluación

Parte 1: Expropiación (35 %)

R1. Traza serial del firmware base, con al menos 4 segundos de ejecución, mostrando el entrelazado de los tres hilos.

R2. Cuarto hilo de mayor prioridad implementado y documentado.

R3. Experimento con prioridad cooperativa realizado, con explicación correcta de por qué cambia el comportamiento.

Parte 2: Inversión de prioridades (45 %)

R4. Medición de latencia de `ALTA` con `USE_MUTEX 0` y con `USE_MUTEX 1`, con evidencia de traza serial para ambos casos.

R5. Explicación correcta de la diferencia, citando herencia de prioridad.

R6. Evidencia de la prioridad efectiva de `BAJA` subiendo y bajando durante la sección crítica en modo mutex.

Parte 3 (opcional): SMP (10 % adicional)

R7. Ejecución exitosa en `qemu_riscv64` con SMP habilitado (o diagnóstico documentado del intento, si no se logra) y análisis de asignación de hilos a *harts*.

Informe (20 %)

R8. Informe PDF (máximo 4 páginas) con las trazas capturadas, las mediciones solicitadas, y las explicaciones de las Actividades de las Partes 1 y 2.

Entregable

- Archivo `ApellidoNombre_Taller08.tar.gz` con:
  1. Código fuente modificado de `threads_demo/` y `priority_inversion/`.
  2. `README.txt`: nombre, carné, si realizó la Parte 3 opcional, instrucciones para compilar y ejecutar en QEMU.
- Informe PDF según lo descrito arriba.

Referencias

[1] M. Jones, "What Really Happened on Mars," *RISKS Digest*, vol. 19, 1997, disponible en: `https://www.cs.unc.edu/ anderson/teach/comp790/papers/mars_pathfinder_long_version.html`.

[2] L. Sha, R. Rajkumar, J. P. Lehoczky, “Priority Inheritance Protocols: An Approach to Real-Time Synchronization,” *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, 1990.

[3] The Zephyr Project, *Zephyr Project Documentation* [en línea], disponible en: `https://docs.zephyrproject.org`, 2024.

[4] The Zephyr Project, *QEMU RISCV64 Board Documentation* [en línea], disponible en: `https://docs.zephyrproject.org/latest/boards/qemu/riscv64/doc/index.html`, 2024.